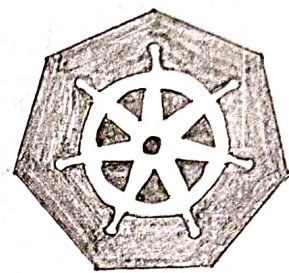


Kubernetes Object

K8's Objects are like instructions or blueprints that tell K8's how your system should look and behave.

They describe:

- What apps should run
- How many copies (replicas) of each app
- What resources (CPU, memory) they need
- How they should connect to each other
- What to do if something crashes



Kubernetes

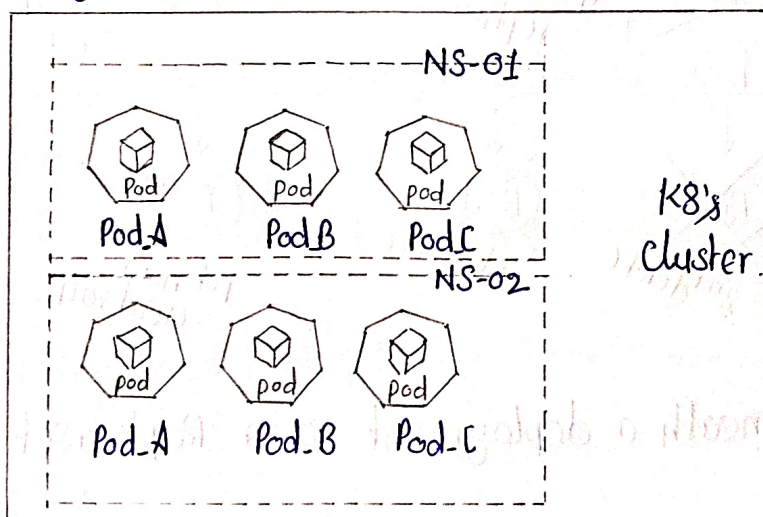
Once you create an object, K8's keeps checking and fixing things to match what you asked for. This is called maintaining the desired state.

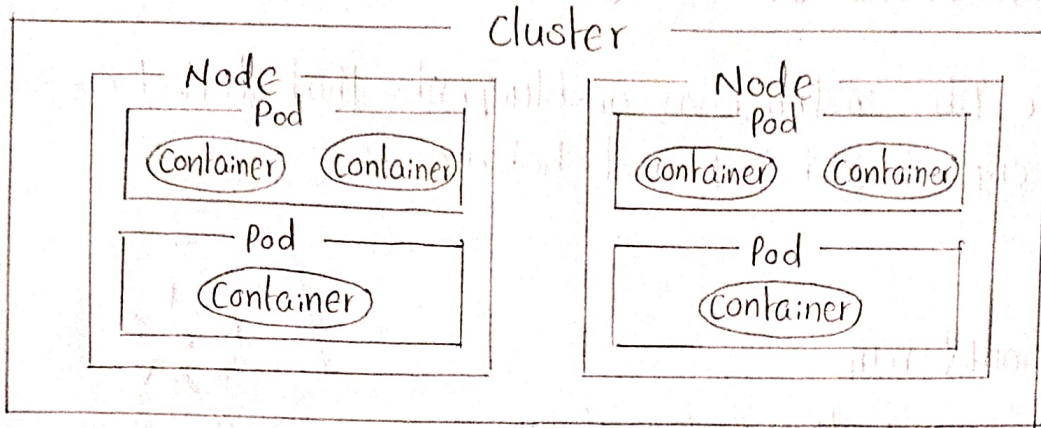
Workload Objects (Run Your Apps)

Pod - The smallest and the most basic object is a pod, which runs one or more containers

→ Containers inside a pod share the same n/w & storage.

→ Pods are temporary, which means if it fails it might get restarted or replaced losing all the pod data. And that's why you normally don't use pods by themselves in production.

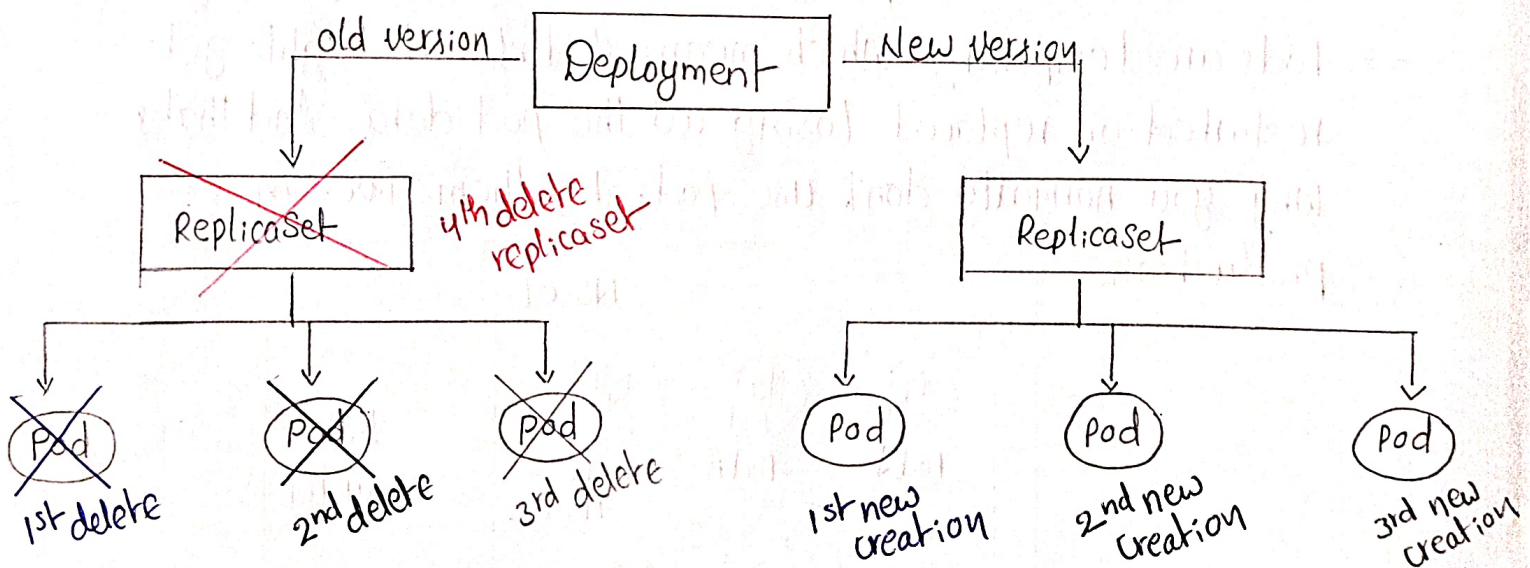




To run pods more reliably, K8's has controllers.

Deployment - A deployment is the most common controller. deployment makes sure that the right number of pods are running.

- creates pods based on a template, maintains desired replicas (e.g., 3 copies of your app).
- Supports scaling up and down.
- Handles rolling updates without downtime, supports rollback if something goes wrong.
- self-heals, if a Pod crashes, it replaces it automatically.



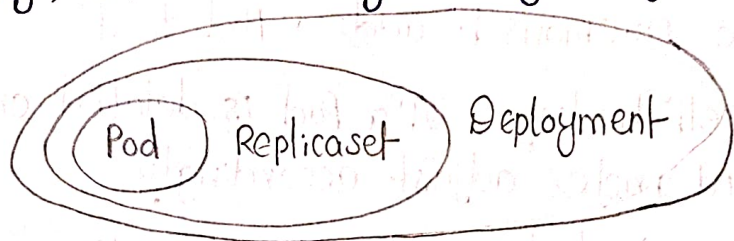
underneath a deployment is a Replicaset.

ReplicaSet - A ReplicaSet maintains a specified number of pod replicas by automatically creating or deleting the pods as needed.

→ Ensures high availability by maintaining the desired number of Pods.

→ Automatically replaces failed Pods.

→ Can be used directly, but is usually managed by a Deployment.



StatefulSet - For application that needs stable storage or Identity like databases, you can use statefulset.

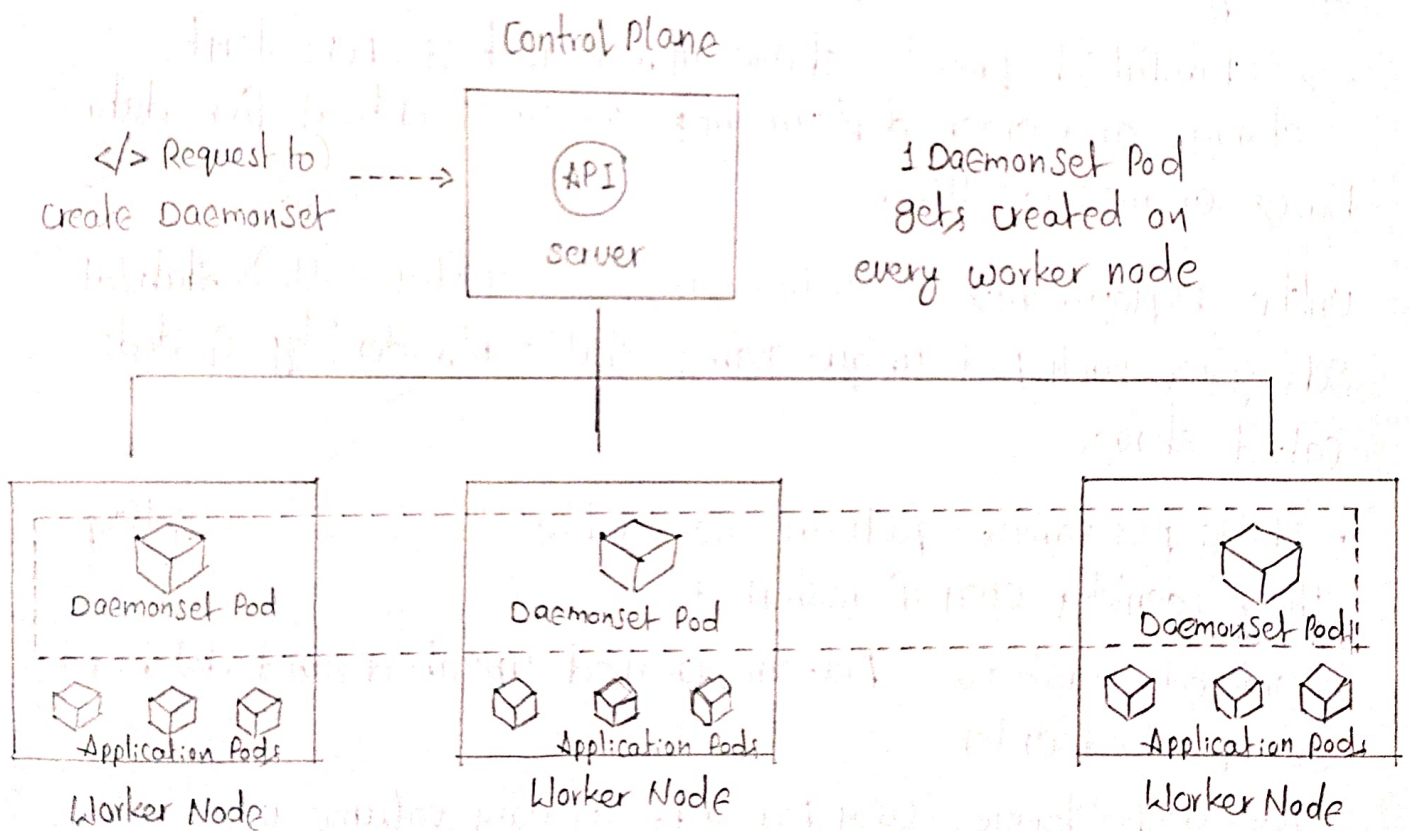
→ statefulset provide stable n/w identities, persistent storage and order deployments which is critical for databases or message use.

Unlike Deployments (which treat Pods as interchangeable), StatefulSets gives each pod unique name, stable n/w identity, & dedicated storage.

- Stable pod names: pods are named like db-0, db-1, db-2 & keep their identity even if restarted.
- Ordered operations: Pods are created, updated, and deleted in specified order.
- Persistent storage: Each Pod gets its own volume via volumeClaimTemplates.
- Ideal for databases: MySQL, MongoDB, Cassandra, Kafka, etc.

Daemonset - for tasks that need to run on every node, such as log collection or monitoring agents, there's daemonset.

- Daemonset ensure that specific pods run on every node or selected nodes.
- one Pod per node - Automatically runs a copy of the Pod on every node.
- Auto-scaling with nodes - if a new node joins the cluster, the Daemonset adds a Pod to it.
- self-healing - If a Pod is deleted or a node is removed, Kubernetes adjust accordingly.
- Target deployment - You can use nodeSelector, affinity, or taint/tolerations to run Pods only on specific nodes.



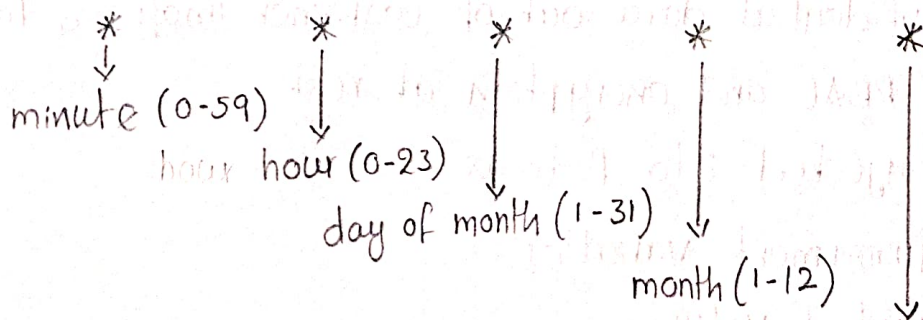
Jobs - if you need to run something once and then stop, you can use a Job.

→ Jobs in K8's are used to run pods that perform finite task and terminate when completed.

- Runs to completion - Unlike Deployment, Jobs don't keep pods running forever.
- Automatic retries - If a Pod fails, K8's will start a new one.
- Tracks success - Job completes only when the specified number of successful runs is reached.
- Supports parallelism - You can run multiple Pods in parallel for batch processing.

CronJob - A CronJob is a Kubernetes resource that runs Jobs on a repeating schedule, similar to linux cron.

- Recurring execution - Runs Jobs at defined intervals.
- Concurrency control - Choose whether to allow overlapping runs (Allow, Forbid, Replace).
- History limits - Track how many successful/failed Jobs to retain.
- Suspend option - Temporarily pause the schedule.



Ex -

00 * * * → Run once a day at midnight

day of the week (0-6)
(Sunday to Saturday)
or
Sun, Mon, Tue, Wed, Thu,
Fri, Sat.

Config & Storage (Manage App Data)

ConfigMap - In k8's ConfigMaps store non-sensitive data like app configuration data as key-value pairs.

→ it helps you separate configuration from application code, making your containers more portable and easier to manage across environments.

- stores config like URLs, file paths, feature flags, etc.
- Can be injected into Pods as:
 - Environment Variables
 - Command-line arguments
 - Mounted files.
- Makes it easy to update settings without rebuilding containers
- Not encrypted - use secrets for sensitive data.

Secret - secret store sensitive data such as password or API keys, SSH keys, TLS certificates.

→ Unlike ConfigMaps, Secrets are designed with security in mind, they're base64-encoded and can be encrypted at rest in etcd.

why use Secrets?

- Keeps confidential data out of container images & Pod specs
- Supports RBAC and encryption at rest.
- Can be injected into Pods as:
 - Environment variables
 - Mounted volumes
 - Image pull credentials

Persistent Volumes (PV) - for storage, k8's uses PV to represent actual disk space either in cloud or physical volumes.

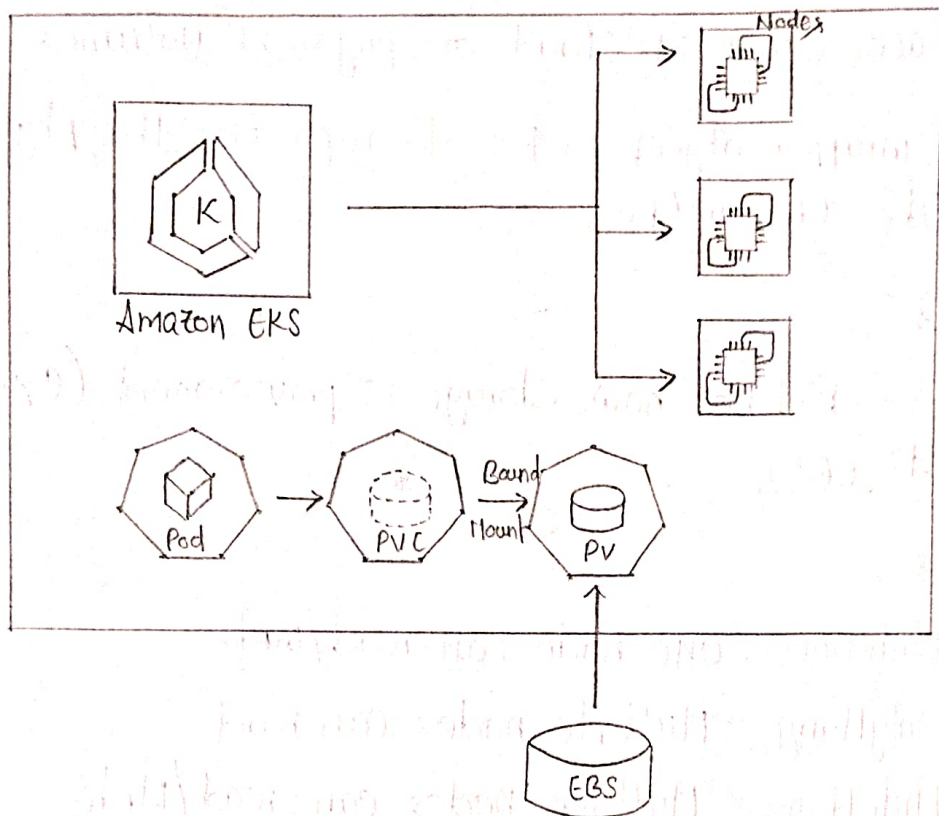
→ PV is the wrapper object in k8's to represent the physical volume like EBS or EFS.

PV components :

- StorageClass - Defines how storage is provisioned (e.g., SSD, NFS, AWS EBS).
- AccessModes -
 - ReadWriteOnce : one node can read/write
 - ReadOnlyMany : Multiple nodes can read
 - ReadWriteMany : Multiple nodes can read/write.
- ReclaimPolicy -
 - Retain : Keeps data after release
 - Recycle : Basic scrub and reuse
 - Delete : Deletes storage after release.

Persistent Volume Claim (PVC) - Use PVC to let pod request disk space on these volumes.

→ it acts as a bridge b/w the application pod & a PV, enabling users to dynamically or statically provision storage resources.



Storage class - A storage class Defines different types of storage like SSDs or slower Disk & allow k8s to automatically provide storage as needed.

→ A storage class in k8s defines how dynamic storage should be provisioned when a Pod requests persistent storage via a PVC.

Think of it like :

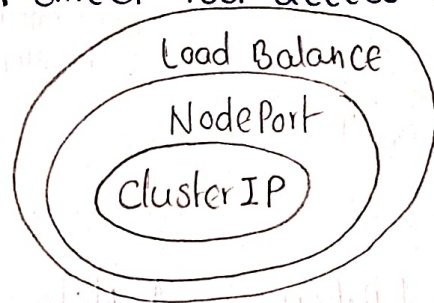
- StorageClass - Menu in a restaurant (defines different dishes → storage types).
- PVC - your order (requesting a specific dish/storage)
- PV - The actual food served (the real storage).

Networking Objects (connect Your Apps)

Service - Service expose pods by giving them permanent IP and DNS name. So, even if pod get replaced, apps can still talk to each other.

Types of services:

- ClusterIP - Default type. Exposes the services Internally within the cluster only.
- NodePort - Exposes the service on a static port on each node's IP.
- LoadBalance - Provisions an external load balancer (e.g., AWS ELB) to expose service
- Headless - No ClusterIP, Used for direct Pod access (e.g., statefulsets, DNS).



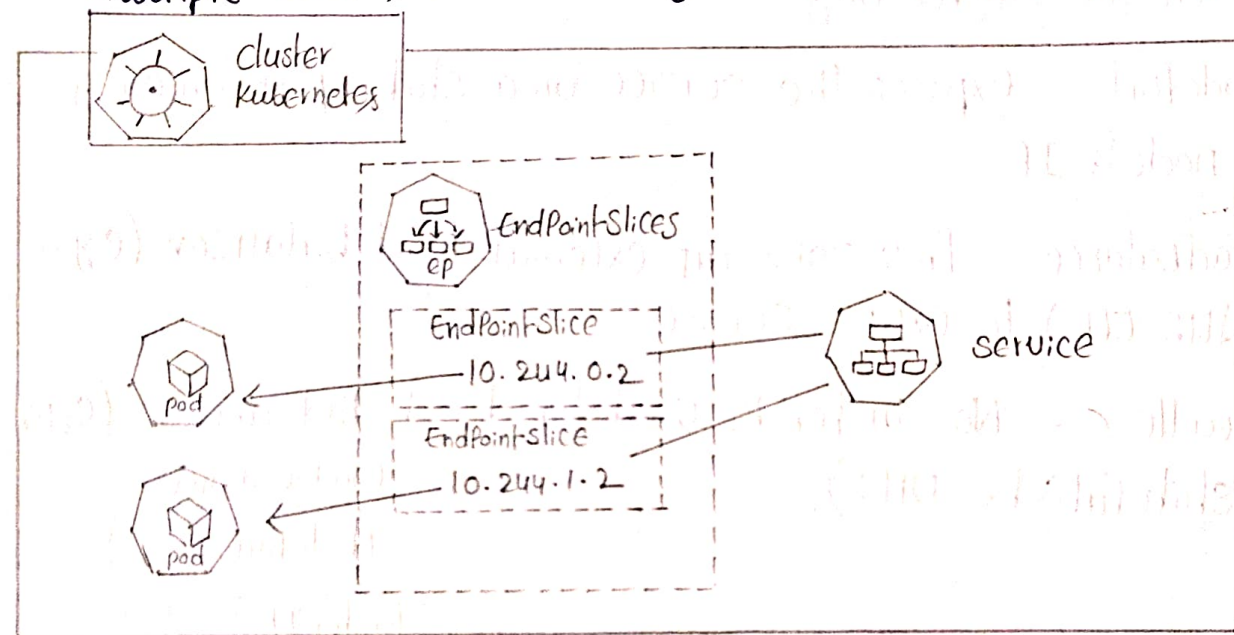
Ingress - Ingress resources manage HTTP and HTTPS traffic routing requests to services based on host names or paths, TLS settings.

Key Components

- Ingress Resource - Defines routing rules (host/path → service)
 - Ingress Controller - Implements the rules (e.g., NGINX, Traefik, ALB)
 - Backend services - The actual apps or API's receiving traffic
- ⚠ Just creating an Ingress resource does nothing unless an Ingress Controller installed.

EndpointSlice - EndpointSlice is a scalable and modern way for Kubernetes to track network endpoint (i.e., Pods) behind a Service.

- Before v1.21, K8's used a single Endpoints objects per service, which became a bottleneck in large cluster.
- EndpointSlice solves this by splitting endpoints into multiple slices, each holding up to 100 endpoints by default.



NetworkPolicy - A NetworkPolicy is a K8's resource that defines rules for how pods can communicate with each other and with external endpoints.

- it's like a firewall for your cluster, controlling traffic at layer 3/4 (IP and port level).
- By default, all pods can talk to each other, once you apply a NetworkPolicy, only explicitly allowed traffic is permitted.

Security & Access Control (Control Access)

Service Account - In K8's, a SA is an identity used by Pods to interact with the Kubernetes API server.

→ By default, when a Pod runs, it automatically gets a default SA unless you assign a custom one.

→ Think of it like a username/password for applications (pods) running inside the cluster.

UserAccount = for humans (developers, Admins)

ServiceAccount = for workloads (Pods, controllers, jobs, etc.).

Role & ClusterRole -

Role : A Role is an RBAC object in K8's.

→ it defines permissions (verbs like get, list, create, delete) on resources (pods, configmaps, secrets, etc).

→ A Role is namespaced → applies only inside a specific namespace.

Ex - Allow reading Pods in dev namespace.

ClusterRole : A ClusterRole is also an RBAC object

→ it defines permissions just like a Role, but it used two cases.

- for non-namespaced resources (like nodes, persistent volumes)
- for cluster-wide access across all namespaces.

Ex - Allow reading Pods in all namespaces.

RoleBinding & ClusterRoleBinding -

RoleBinding : A RoleBinding is used to grant permissions (from a Role or ClusterRole) to a user, group, or ServiceAccount within a specific namespace.

→ Even if you bind a ClusterRole with a RoleBinding, the permissions are limited to that namespace.

Ex - Allow user cherry to read Pods only in the dev namespace.

ClusterRoleBinding : A ClusterRoleBinding is used to grant permissions (from a ClusterRole) cluster-wide.

→ it applies across all namespaces.

Ex - Allow user togorel to read Pods in all namespaces.

Cluster Management & Scheduling (Optimize Resources)

Namespaces - A Namespace is a logical partition inside a K8's cluster.

- it helps organize & isolate resources (Pods, Services, ConfigMaps, Secrets, etc.) for different teams or environments.
- think of it like folders in an OS (or) apartments in the same building.

Why do we use NS ?

- Multi-tenancy - Different teams (dev, QA, prod) can share the same cluster without interfering.
- Resource isolation - You can apply ResourceQuota and LimitRange per namespace.
- Access control - Use RBAC (Roles/RoleBindings) at namespace level.
- Organization - Makes it easier to manage resource in large clusters.

ResourceQuota - ResourceQuota limit how much CPU, Memory, or Storage a Namespace can use.

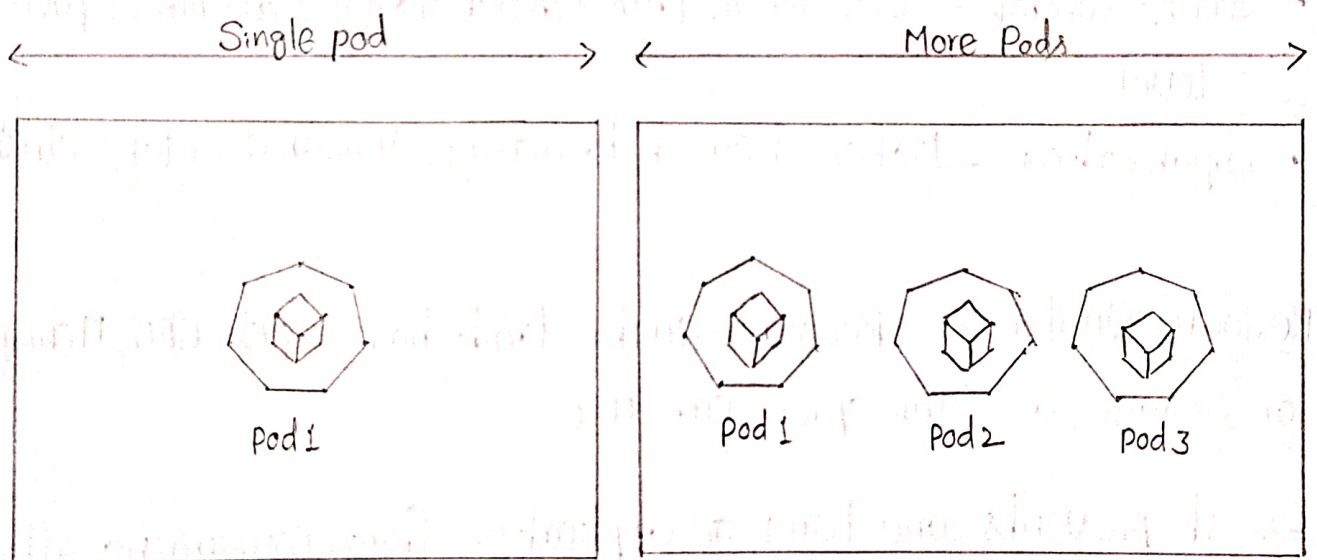
- it prevents one team or application from consuming all cluster resources, ensuring fair sharing, stability, and cost efficiency.

LimitRange - A LimitRange sets minimum, maximum, and default resource requests/limits (CPU, memory, storage) for Pods or Containers in a namespace.

→ it ensures workloads don't over-consume resources and that every Pod requests a reasonable amount.

Horizontal Pod Autoscaler (HPA) - HPA that automatically scales the number of Pod replicas in a Deployment, ReplicaSet, or StatefulSet based on resource usage (CPU, memory) or custom metrics.

→ Scaling is done horizontally - by adding/removing Pods (not changing Pod size).

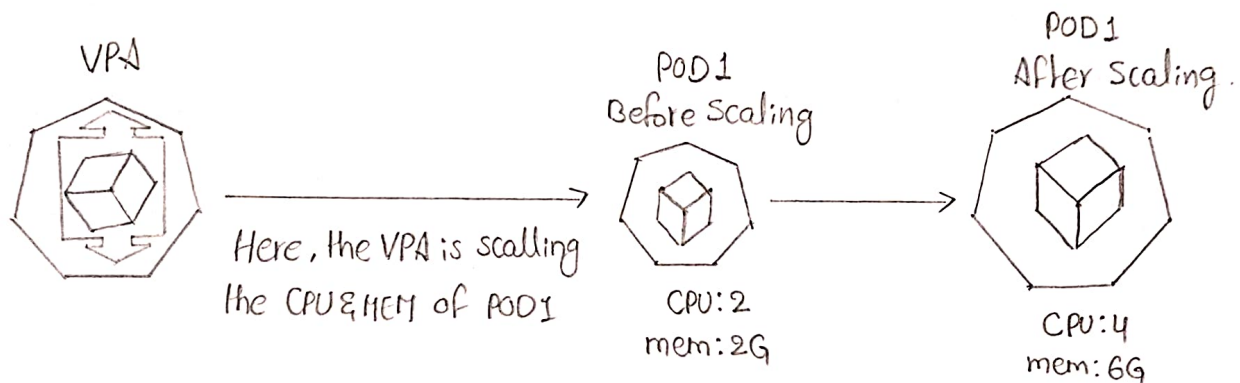


NODE1 - Before HPA scaling

NODE1 - After HPA scaling.

Vertical Pod Autoscaler (VPA) - VPA that automatically adjust the CPU and memory requests/limits for your Pods based on actual usage.

→ Unlike HPA (Which scales by adding/removing Pods), VPA resizes Pods vertically - gives the more (or less) resources.



PodDisruptionBudget (PDB) - A PDB is a policy in K8's that ensures a minimum number of Pods (or percentage) in an application remain running during voluntary disruption.

→ Voluntary disruption = actions initiated by humans or automation (e.g., node drain, cluster upgrade, scaling down).

→ it helps maintain high availability of critical applications.

⚠ Note: PDB does not protect against involuntary disruptions like node crashes, kernel panics, or hardware failures.